Vídeo 4.3: Esperar sem travar (código assíncrono)

Duração-alvo: 6 min | **Conceitos:** assíncrono, promessa, async e await | **O aluno sai sabendo:** prever a ordem de execução de um código simples com await.

Antes de gravar: criar um arquivo assincrono.js, separado do servidor, para a demonstração.

[0:00] Abertura

Na tela: título do vídeo.

Fala: “Antes de conectar o banco, tem um conceito que quase todo mundo estranha no começo: o código assíncrono. Parece complicado, mas com uma analogia simples dá para entender. E sem ele não dá para usar um banco de dados.”

[0:40] O depósito demorado

Na tela: ilustração de uma atendente que pede um livro ao depósito e, enquanto espera, atende outra pessoa.

Fala: “Na biblioteca, alguns livros ficam em um depósito no subsolo. Quando você pede um deles, a atendente avisa o depósito, e a entrega demora alguns minutos. O que a atendente faz? Se ficasse parada olhando para a porta, a fila inteira esperaria. Ela atende a próxima pessoa e, quando o livro chega, entrega. Isso é o assíncrono: pedir uma coisa que demora e continuar trabalhando enquanto ela não fica pronta. O banco de dados é o nosso depósito. Ler e gravar no disco demora um pouquinho, e o servidor não deve parar de atender os outros clientes nesse tempo.”

[2:00] Vendo na prática

Na tela: no assincrono.js, escrever o código abaixo, com o cursor destacando cada parte durante a explicação.

```
function esperar(ms) {
  return new Promise((resolve) => setTimeout(resolve, ms));
}

async function buscarDoDeposito() {
  console.log('1. Pedi o livro ao depósito...');
  await esperar(2000);
  console.log('2. O livro chegou!');
}

buscarDoDeposito();
console.log('3. Atendendo outra pessoa enquanto espera');
```

Fala: “A função esperar simula o depósito: ela leva o tempo que a gente pedir, em milissegundos. Não precisa entender como ela foi escrita por dentro. O que importa é que ela devolve uma promessa, que em inglês é Promise. Promessa é como a senha que a atendente te dá: um comprovante de que o livro vai chegar. A função buscarDoDeposito tem a palavra async antes dela, e dentro dela usamos await. Await quer dizer: espere a

promessa se cumprir antes de passar para a próxima linha dessa função. Só dessa função. O resto do programa não espera.”

[3:30] Rodando

Na tela: no terminal, rodar `node assincrono.js` e mostrar a saída.

```
1. Pedi o livro ao depósito...
 3. Atendendo outra pessoa enquanto espera
 2. O livro chegou!
```

Fala: “Vou rodar. Olha a ordem: um, três, dois. O programa fez o pedido, não ficou parado, atendeu o resto e, dois segundos depois, o livro chegou. Se tudo acontecesse na ordem do arquivo, o três só apareceria depois do dois.”

[4:30] O que muda no banco

Na tela: trocar a linha do `await` por `console.log(esperar(1000));` e mostrar que aparece `Promise { <pending> };` depois desfazer.

Fala: “No banco de dados vai ser igualzinho. Cada comando, como listar ou gravar livros, devolve uma promessa. Nós escreveremos await na frente, e a função que contém o await ganha a palavra async. Duas regras para guardar. Primeira: await só funciona dentro de uma função async. Segunda: esquecer o await é o erro mais comum. Olha o que aparece se eu imprimir a promessa sem esperar: Promise pending, ou seja, promessa pendente. O programa seguiu adiante com a senha na mão, em vez do livro.”

[5:30] Resumo

Na tela: três frases: assíncrono, promessa, async e await.

Fala: “Resumindo: assíncrono é pedir algo que demora e continuar trabalhando. Uma promessa é a senha de que a resposta vai chegar. Com await, a gente espera a promessa dentro de uma função async. No próximo vídeo, vamos usar tudo isso para conectar a nossa API ao SQLite.”

Exercício sugerido: mudar o tempo de espera, acrescentar uma quarta mensagem depois do `await` e prever a ordem das mensagens antes de rodar.